Slip 05

Q1) A Write short notes on network security threats, including Man-In-The-Middle (MITM) and Packet Sniffing attacks.

Q1) A) Write short notes on network security threats, including Man-In-The-Middle (MITM) and Packet Sniffing attacks.

1. Network Security Threats

Network security threats are activities that attempt to damage a network, steal confidential information, or gain unauthorized access to computer systems and data.

Common types of network security threats:

1. Malware: Malicious software such as viruses, worms, and Trojans that damages systems or steals data.
2. Phishing: A technique used to trick users into revealing passwords, banking details, or other sensitive information.
3. Denial of Service (DoS): An attack that overloads a server with requests, making services unavailable to legitimate users.
4. Man-In-The-Middle (MITM): An attacker secretly intercepts communication between two parties.
5. Packet Sniffing: Capturing and analyzing data packets transmitted over a network.

2. Man-In-The-Middle (MITM) Attack

A Man-In-The-Middle attack occurs when an attacker secretly intercepts or modifies communication between two parties without their knowledge.

Example: A user connects to an unsecured public Wi-Fi network to access a banking website. An attacker intercepts the communication and may attempt to steal login credentials.

Working of a MITM attack:

1. The user sends data to the intended server.
2. The attacker intercepts the communication.
3. The attacker may read or modify the transmitted data.
4. The communication continues, often without either party noticing.

Prevention:

- Use HTTPS-secured websites.
- Avoid sensitive transactions on unsecured public Wi-Fi.
- Use a trusted VPN when appropriate.
- Enable multi-factor authentication (MFA).

3. Packet Sniffing Attack

Packet sniffing is the process of capturing and examining data packets transmitted over a computer network. Attackers may use packet-sniffing tools to monitor network traffic and attempt to obtain sensitive information.

Example: When a user sends unencrypted login details over an insecure network, an attacker monitoring that network may capture and read those details.

Types of packet sniffing:

1. Passive sniffing: The attacker monitors network traffic without actively changing it.
2. Active sniffing: The attacker manipulates network traffic or network operations to capture packets, often on switched networks.

Prevention:

- Use encryption protocols such as HTTPS and TLS.
- Avoid sending confidential information over unsecured networks.
- Use secure Wi-Fi encryption, such as WPA2 or WPA3.
- Monitor network traffic for suspicious activities.

Conclusion

Network security threats can compromise the confidentiality, integrity, and availability of data. MITM attacks intercept or modify communication, whereas packet sniffing captures and analyzes network packets. Encryption, secure connections, authentication, and network monitoring help protect against these threats.

Q2) B Write a Python program to demonstrate IP tracking (simulation):Store fake IP addresses and Print location mapping (use dictionary)

```
# IP Tracking Simulation using Dictionary

ip_location = {
```

```
    "192.168.1.10": {"city": "Pune", "country": "India"},
    "192.168.1.20": {"city": "Mumbai", "country": "India"},
    "10.0.0.5": {"city": "Delhi", "country": "India"},
    "172.16.0.8": {"city": "Nagpur", "country": "India"},
    "192.168.2.15": {"city": "Chennai", "country": "India"}
}

print("Available IP Addresses:")
for ip in ip_location:
    print(ip)

target = input("\nEnter an IP address to track: ")

if target in ip_location:
    print("\nIP Address:", target)
    print("City:", ip_location[target]["city"])
    print("Country:", ip_location[target]["country"])
    print("Tracking Status: Location Found")
else:
    print("IP address not found in database!")
```

OR

Q2) B Write a simple Python program to hide a text message inside an image using steganography and display the hidden message again

```
from PIL import Image
import stepic


# Function to hide message
def hide_data():

    # Ask user for original image
    image_name = input("Enter the original image name: ")

    # Open image
    img = Image.open(image_name)

    # Ask user for secret message
```

```
    secret_msg = input("Enter the secret message: ")

    # Convert message into bytes
    data = secret_msg.encode("utf-8")

    # Hide message inside image
    stego_img = stepic.encode(img, data)

    # Ask for output image name
    output_name = input("Enter name for stego image (use .png):
")

    # Save as PNG
    stego_img.save(output_name, format="PNG")

    print("Message successfully hidden!")
    print("Stego image:", output_name)


# Function to retrieve message
def retrieve_data():

    # Ask user for stego image
    image_name = input("Enter the stego image name: ")

    # Open stego image
    stego_img = Image.open(image_name)

    # Decode hidden data
    decoded_data = stepic.decode(stego_img)

    # Convert bytes to normal text
    print("Retrieved Message:", decoded_data.decode("utf-8"))


# Main program
hide_data()
retrieve_data()
```